

```
return meta, limite

class AgregadosEngine:
    """
    Motor principal de cálculo de agregados.
    Cada método hace un commit por lote, no por renglón.
    """

    VENTANAS = (7, 14, 30)

    def calcular_ventanas_usuario(self, usuario_id: int, fecha_fin: date) -> Dict[str, int]:
        logger.info(f"[AGG] Ventanas user={usuario_id} fecha_fin={fecha_fin}")
        cont = {"ventanas": 0, "categorias": 0}

        for dias in self.VENTANAS:
            f_ini, f_fin = _rango(fecha_fin, dias)
            por_cat = _minutos_por_categoria(usuario_id, f_ini, f_fin)
            dias_cat = _dias_con_datos_por_categoria(usuario_id, f_ini, f_fin)
            total = _total_global(usuario_id, f_ini, f_fin) or 1.0

            for cat, mins in por_cat.items():
                dias_con = int(dias_cat.get(cat, 0)) or 1
                obj = AggVentanaCategoria(
                    usuario_id=usuario_id,
                    categoria=cat,
                    ventana=f"{dias}d",
                    fecha_fin=fecha_fin,
```

Examen C1

05/02/2026

Guzmán López Ángel Eduardo

Luis Ali Rios Cristobal

Rodolfo Reyes Ramirez

Universidad Politécnica de Chiapas

5-C

Base de datos avanzadas - Ramsés Camas Nájera

¿Por qué escogieron ese caso?

Se eligió el caso 3 ya que, al analizar los casos posibles dimos con la conclusión que nuestro problema se asimila más a una relación estándar. Esto mismo hace que sea más fácil cumplir las relaciones de una forma más sencilla sin necesitar de una lógica de negocio muy complicada.

¿Por qué lo diseñaron así?

Primero modelamos las cosas físicas como el Centro de Convenciones, salones, ponentes etc. Para después crear el atributo de Capacidad en la tabla Salon, esto es crucial para la integridad de nuestra base de datos al saber el límite de personas antes de cualquier inscripción.

La relación entre Evento y Sesión el cual sería 1 a muchos ya que un evento contiene su fecha de inicio y fin, pero esto ocurre dentro de Sesiones (una plática, un taller, etc). así que aquí se tomó la decisión de agregar un cupo máximo en la sesión. Esto en caso de querer limitar un taller de 80 lugares a 40 personas.

Modelamos una tabla intermedia en la cual definimos la relación entre un ponente y una sesión ya que un mismo ponente puede tener múltiples sesiones y para poder validar de manera efectiva las sesiones que tiene un ponente interceptamos estas dos informaciones en la tabla ponente sesión

Modelamos los pagos como una entidad débil de paquetes usando el borrado en cascada en caso de que se elimine un paquete se borren asimismo sus registros de abonos además de implementar el uso de Triggers para asegurar que no se ingrese una cantidad de dinero mayor a la que vale el paquete.

tuvimos una tabla intermedia Inscripción con un atributo booleano Asistió, en caso de que una persona se registre pero esta no haya llegado. Esto es vital para la sección de reportes ya que sin este campo no podríamos ver la diferencia entre el interés del evento y las asistencias reales.

Separamos Patrocinios de Paquetes ya que cada uno cuenta con un ingreso de dinero pero cada uno cuenta con una lógica diferente. Un paquete implica ciertos servicios mientras que un Patrocinio implica marketing, al separarlos evitamos así tener una tabla enorme de información nula, mantenemos ambas tablas fijas y las unimos mediante el uso de nuestras Views.

Como sabemos un asistente puede ir a diferentes sesiones por lo que diseñamos la relación de uno a muchos para que un mismo usuario/asistente pueda asistir a múltiples sesiones

validando su asistencia en esa tabla y a su vez la tabla inscripcion cuenta con una relación de muchas inscripciones a una sola sesión (N:1)

Documentación de Vistas (VIEWS)

Explicación de cada una de las vistas creadas para el sistema de gestión de eventos corporativos.

1. Vista: Resumen Financiero por Evento

¿Qué hace?

Esta vista muestra un panorama completo de los ingresos de cada evento. Calcula cuánto dinero se ha recibido por la venta de paquetes corporativos y cuánto por patrocinios. También indica qué porcentaje del ingreso total proviene de los patrocinadores.

¿Por qué es útil

Permite al equipo financiero saber de un vistazo cuánto ha generado cada evento y qué tan dependiente es de los patrocinadores. Si un evento no tiene patrocinadores, la vista lo maneja correctamente sin generar errores.

```
```sql
SELECT
 e.id_evento,
 e.nombre AS evento,
 COALESCE(paq.ingresos_paquetes, 0) AS ingresos_paquetes,
 COALESCE(pat.ingresos_patrocinios, 0) AS ingresos_patrocinios,
 COALESCE(paq.ingresos_paquetes, 0) + COALESCE(pat.ingresos_patrocinios, 0) AS ingreso_total,
 ROUND(
 (COALESCE(pat.ingresos_patrocinios, 0) * 100.0) /
 NULLIF(COALESCE(paq.ingresos_paquetes, 0) + COALESCE(pat.ingresos_patrocinios, 0), 0), 2
) AS porcentaje_patrocinios
FROM Evento e
LEFT JOIN (...) paq ON e.id_evento = paq.id_evento
LEFT JOIN (...) pat ON e.id_evento = pat.id_evento;
```
```

2. Vista: Asistencia por Sesión

¿Qué hace?

Muestra estadísticas de asistencia para cada sesión del evento. Indica cuántas personas se inscribieron, cuántas realmente asistieron, y calcula la tasa de asistencia. Además, clasifica qué tan llena estuvo la sesión respecto a su cupo máximo.

¿Por qué es útil?

Ayuda a los organizadores a identificar qué sesiones fueron más populares y cuáles tuvieron problemas de ausentismo. La clasificación de ocupación (Llena, Alta, Media, Baja, Muy Baja) permite tomar decisiones para futuros eventos.

```

```sql
SELECT
 s.nombre AS sesion,
 s.tipo,
 s.cupo_maximo,
 COUNT(i.id_asistente) AS total_inscritos,
 SUM(CASE
 WHEN i.asistio = TRUE THEN 1
 ELSE 0
 END) AS total_asistentes,
 ROUND(
 (SUM(CASE
 WHEN i.asistio = TRUE THEN 1
 ELSE 0
 END) * 100.0) /
 COUNT(i.id_asistente), 2
) AS tasa_asistencia,
 CASE
 WHEN (COUNT(i.id_asistente) * 100.0 / s.cupo_maximo) >= 100 THEN
 'Llena' WHEN (COUNT(i.id_asistente) * 100.0 / s.cupo_maximo) >= 75 THEN 'Alta'
 WHEN (COUNT(i.id_asistente) * 100.0 / s.cupo_maximo) >= 50 THEN 'Media'
 ELSE 'Baja'
 END AS clasificacion_ocupacion
FROM Sesion s
LEFT JOIN Inscripcion i ON s.id_sesion = i.id_sesion
GROUP BY s.id_sesion, s.nombre, s.tipo, s.cupo_maximo;

```

### 3. Vista: Ponentes Más Activos

¿Qué hace?

Lista a los ponentes ordenados por su nivel de participación. Muestra cuántas sesiones ha impartido cada uno, en cuántos eventos distintos ha participado, y cuál es el rol que más frecuentemente desempeña (presentador, panelista, moderador, instructor).

¿Por qué es útil?

Permite identificar a los ponentes más comprometidos y versátiles. Es útil para la planificación de futuros eventos y para reconocer a colaboradores destacados.

```

```sql
SELECT
  CONCAT(p.titulo, ' ', p.nombre, ' ', p.apellido) AS ponente,
  p.especialidad,
  COUNT(ps.id_sesion) AS total_sesiones,
  COUNT(DISTINCT s.id_evento) AS eventos_distintos,
  (
    SELECT ps2.rol
    FROM Ponente_Sesion ps2
    WHERE ps2.id_ponente = p.id_ponente
    GROUP BY ps2.rol
    ORDER BY COUNT(*) DESC
    LIMIT 1
  ) AS rol_mas_frecuente
FROM Ponente p
JOIN Ponente_Sesion ps ON p.id_ponente = ps.id_ponente
JOIN Sesion s ON ps.id_sesion = s.id_sesion
GROUP BY p.id_ponente
HAVING COUNT(ps.id_sesion) >= 1
ORDER BY total_sesiones DESC;

```

4. Vista: Inversión por Empresa Cliente

¿Qué hace?

Muestra un resumen de la inversión que cada empresa cliente ha realizado. Incluye el número de paquetes comprados, el monto total que han pagado, cuántos asistentes han enviado, y el costo promedio por cada asistente.

¿Por qué es útil?

Permite identificar a los clientes más valiosos y analizar la relación costo-beneficio desde la perspectiva de las empresas. Ayuda en la toma de decisiones para ofrecer descuentos o paquetes personalizados.

```

```sql
SELECT
 ec.razon_social,
 ec.rfc,
 COUNT(DISTINCT p.id_paquete) AS total_paquetes,
 COALESCE(SUM(pg.monto_pagado), 0) AS monto_total_pagado,
 COUNT(DISTINCT a.id_asistente) AS total_asistentes,
 CASE
 WHEN COUNT(DISTINCT a.id_asistente) > 0
 THEN ROUND(SUM(pg.monto_pagado) / COUNT(DISTINCT a.id_asistente), 2)
 ELSE 0
 END AS costo_promedio_asistente
FROM EmpresaCliente ec
LEFT JOIN Paquete p ON ec.id_empresa = p.id_empresa
LEFT JOIN Pago pg ON p.id_paquete = pg.id_paquete
LEFT JOIN Asistente a ON p.id_paquete = a.id_paquete
GROUP BY ec.id_empresa, ec.razon_social, ec.rfc
ORDER BY monto_total_pagado DESC;

```

## 5. Vista: Balance de Pagos por Paquete

¿Qué hace?

Muestra el estado financiero de cada paquete vendido. Para cada paquete indica su costo total, cuánto se ha pagado hasta el momento, cuánto falta por pagar (saldo pendiente), y clasifica el estado del pago.

Estados de pago:

- Sin Pago: No se ha recibido ningún pago
- Parcial: Se han hecho pagos pero no cubren el total
- Completo: El paquete está completamente pagado
- Cancelado: El paquete fue cancelado

¿Por qué es útil?

Permite al área de cobranza identificar rápidamente qué paquetes tienen saldos pendientes y dar seguimiento a los pagos. También ayuda a detectar paquetes cancelados que podrían requerir reembolsos.

```
```sql
SELECT
    ec.razon_social AS empresa,
    e.nombre AS evento,
    p.tipo_paquete,
    p.costo_total,
    COALESCE(pg.total_pagado, 0) AS total_pagado,
    p.costo_total - COALESCE(pg.total_pagado, 0) AS saldo_pendiente,
    CASE
        WHEN p.estado = 'Cancelado' THEN 'Cancelado'
        WHEN COALESCE(pg.total_pagado, 0) = 0 THEN 'Sin Pago'
        WHEN COALESCE(pg.total_pagado, 0) >= p.costo_total THEN 'Completo'
        ELSE 'Parcial'
    END AS estado_pago
FROM Paquete p
JOIN EmpresaCliente ec ON p.id_empresa = ec.id_empresa
JOIN Evento e ON p.id_evento = e.id_evento
LEFT JOIN (
    SELECT id_paquete, SUM(monto) AS total_pagado
    FROM Pago WHERE estado = 'Completado'
    GROUP BY id_paquete
) pg ON p.id_paquete = pg.id_paquete;
```